From LFSRs to LFGs: Periodicity and Structural Transformations in Stream Ciphers

Shivarama K. N[1]., Susil Kumar Bishoi[2], Vadiraja Bhatta G. R.[3], Vashek Matyas[4]
[1,3]Department of Mathematics, Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, India - 576104.
[2]Center for Artificial Intelligence and Robotics, Defence Research and Development Organisation, CV Raman Nagar, Bengaluru India-560093,
[4]Masaryk University, Brno, Czech Republic,
E-mail addresses of contributing authors:shivaramakn1@gmail.com; skbishoi.cair@gov.in; matyas@fi.muni.cz

Abstract

Feedback shift registers, such as Linear Feedback Shift Registers (LFSRs), Multi-Recursive Matrix Methods (MRMMs), and Lagged Fibonacci Generators (LFGs), are fundamental components in stream cipher-based cryptographic systems. In this paper, we investigate systems composed of LFSRs under two distinct configurations. First, we study the cascade connection of LFSRs and demonstrate that it represents a special case of the first configuration. Under specific conditions, we derive the exact period of these cascaded systems. Second, we analyze a system comprising two LFSRs in the second configuration, where carry bits are introduced into the feedback computation of the second LFSR. We examine the periodicity of both the carry bits and the overall system. Furthermore, we generalize this construction to word size m, and show that an additive LFG can be represented by an equivalent system of LFSRs. This approach enables efficient LFG implementation in resource-constrained environments by using multiple LFSRs and a simple adder, thus eliminating the need for large word sizes.

Keywords: Linear Feedback Shift Register, Additive Lagged Fibonacci Generator, Multi-Recursive Matrix Method, Periodicity, Stream Cipher

1 Introduction

Fibonacci numbers were first described in Indian mathematics as early as 200 BC [1]. The famous sequence was published in 1202 by famous Leonardo Fibonacci [2]. The Fibonacci sequence is the sequence of infinite numbers where each number is the sum of the two preceding numbers. This sequence may be defined by the recurrence relation $F_n = F_{n-1} + F_{n-2}$ for $n > 1$ with initial numbers $F_0 = 0$ and $F_1 = 1$. One simple generalization is $F_n = F_{n-j} \star F_{n-k} \pmod{M}$, where the operator $\star$ denotes a general binary operation such as addition and multiplication. Here, the fixed positive integers j and k used in the recurrence relation are called lags with $k > j$. In this generalized recurrence relation, each new term is computed using only two previous terms. However, the number of previous terms involved can be greater than two, leading to what is known as an LFG [3, 4]. An LFG is

a type of pseudorandom number generator (PRNG) that produces pseudorandom numbers with improved statistical properties. If the operation used $\star$ is addition, then it is known as an Additive LFG (ALFG) and if multiplication is used, it is a Multiplicative LFG (MLFG). Usually, m is a power of 2 (i.e., $M = 2^m$) where m is the word size of the processor. The maximum period of LFG depends on the binary operation $\star$ and the characterstic polynomial of LFG. If the degree of the characterstic polynomial is n, then the order of LFG is n. In this case, the maximum period of ALFG and MLFG is $(2^n - 1)2^{m-1}$ and $(2^n - 1)2^{m-3}$, respectively [4].

Lagged Fibonacci Generators are now widely used in cryptography, simulations, and other domains that require efficient generation of pseudorandom sequences. Their appeal lies in their ability to produce long, non-repeating periods and their computational efficiency. By tuning parameters such as the order and feedback function, LFGs can achieve sequences with desirable statistical properties, including uniformity and unpredictability [2]. M. Mascagni and A. Srinivasan have contributed significantly to this field by developing scalable, parallel implementations of several pseudorandom number generators, including LFGs [5]. Their work features parameterized versions of LFGs optimized for high-performance computing environments. Moreover, LFGs are well-suited for high-speed implementations, as they leverage the capabilities of modern word-based processors [6].

ALFG becomes an LFSR when the word size is taken as 1. The concept of the LFSR traces back to early digital systems and telecommunications, where simple shift-register architectures with XOR-based feedback were employed for sequence generation, while its formal mathematical underpinnings were developed in the mid-20^{th} century. Today, LFSRs enjoy ubiquitous deployment across both high-performance and resource-constrained environments owing to their simplicity, efficiency, and versatile bit-sequence properties. Consequently, they are integral to hardware and software systems for applications such as pseudo-random number generation, cyclic redundancy checks and error-detection, radio jamming and noise generation, and FPGA-based PRNGs in IoT or quantum-safe systems.

LFSR is typically a bit-oriented feedback shift register and it does not take advantage of the available modern word-based processors. Instead the word-oriented primitive MRMM [7, 6], and LFG take this advantage.

In this paper, we revisit LFSRs, MRMMs, and Additive Lagged Fibonacci Generators (ALFGs), and explore the connection between ALFGs and LFSRs as discussed in [8, 9]. We also derive certain randomness properties for systems composed of LFSRs and MRMMs.

The structure of the paper is as follows:

- In Section 2, we introduce the necessary notation and definitions, followed by a review of three well-known feedback shift registers (FSRs) and some of their key results.
- In Section 3, we investigate systems composed of multiple LFSRs under two distinct configurations and show how the additive LFG can be expressed in terms of LFSRs.

- In Section 4, we examine the periodicity and perform a comparative study of one of the key statistical properties—avalanche effect—in three MRMM-based cascade systems.
- Finally, we present our conclusions in Section 5.

2 Preliminaries

Let $\mathbb{F}_2$ denote the finite field with two elements, and let $\mathbb{F}_2^m$ represent an m-dimensional vector space over $\mathbb{F}_2$. We use the symbol $\oplus$ to denote addition in $\mathbb{F}_2$, whereas $+$ denotes addition in the residue class ring $\mathbb{Z}/2^m\mathbb{Z}$. Let $M_m(\mathbb{F}_2)$ denote the set of all $m \times m$ matrices over $\mathbb{F}_2$, and let $\mathrm{GL}_m(\mathbb{F}_2)$ be the group of invertible matrices in $M_m(\mathbb{F}_2)$. The identity matrix of size $m \times m$ is denoted by I_m. For any matrix $C \in M_m(\mathbb{F}_2)$, we write $\det(C)$ for its determinant, and $\det(C - xI_m)$ for its characteristic polynomial, which is a degree-m polynomial over $\mathbb{F}_2$. The *order* of C, denoted $O(C)$, is defined as the smallest positive integer k such that $C^k = I_m$. If no such integer k exists, we define $O(C) = 0$.

It is known that the finite field $\mathbb{F}_{2^m}$ is isomorphic to the vector space $\mathbb{F}_2^m$ [7]. Consequently, any element of $\mathbb{F}_{2^m}$ can be represented as a column vector of length m over $\mathbb{F}_2$. This allows matrix-vector multiplication $C\mathbf{s}$ to be interpreted as an element of $\mathbb{F}_2^m$ for any $\mathbf{s} \in \mathbb{F}_{2^m}$ and $C \in M_m(\mathbb{F}_2)$.

Throughout the paper, we adopt the convention that boldface letters such as $\boldsymbol{x}$ denote elements of $\mathbb{F}_2^m$, while regular letters like x refer to individual bits in $\mathbb{F}_2$. The Hamming weight of $\boldsymbol{x}$ denoted by $Wt(\boldsymbol{x})$, defined as the number of nonzero entries in binary expression of $\boldsymbol{x}$.

Suppose, $\mathcal{MP}_m[\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{n-1}] = M_m(\mathbb{F}_2)[\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{n-1}]/({\boldsymbol{x}_0}^2 \oplus \boldsymbol{x}_0, \ldots, {\boldsymbol{x}_{n-1}}^2 \oplus \boldsymbol{x}_{n-1})$ denote the set of all multivariate polynomial $F(.)$ in n variables $\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{n-1}$ with coefficients in $M_m(\mathbb{F}_2)$ such that $F(\mathbf{0}, \ldots, \mathbf{0}) = \mathbf{0}$ and each $\boldsymbol{x}_i \in \mathbb{F}_{2^m}$. If $F \in \mathcal{MP}_m[\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{n-1}]$, then it can be expressed as follows.

$$F(\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{n-1}) = \sum_{I \in P(N)} C_I \prod_{k \in I} \boldsymbol{x}_k, \tag{1}$$

where $P(N)$ denotes the power set of $N = \{0, \ldots, n-1\}$ and $C_I \in M_m(\mathbb{F}_2)$. In Eq. (1), $\mathbf{X} = \prod_{k \in I} \boldsymbol{x}_k$ is computed first, which returns an m-bit number and then $C_I \boldsymbol{X}$ is calculated using matrix-vector multiplication. Here the product $\prod$ can be either field multiplication or modular integer multiplication $*$ or bitwise AND operation &. Similarly, the sum $\sum$ is either modular integer addition $+$ or bitwise XOR operation $\oplus$. As field multiplication is an expensive operation compared to the other two, so we only focus on the other two multiplication operations $*$ and &. The algebraic degree of F denoted as $deg(\mathrm{F})$ can be defined as $\max\{|I| : C_I \neq 0\}$, where $|I|$ denotes the size of I.

A Word-oriented FSR (WFSR) with n stages is a type of an FSR in which each stage holds a word of m bit size. For the sake of convenience, let the initial

state of WFSR be $\mathbf{S}_0 = (\mathbf{s}_0, \mathbf{s}_1, \ldots, \mathbf{s}_{n-1})$ and assume the feedback function $F \in \mathcal{MP}_m[\boldsymbol{x}_0, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_{n-1}]$. At each clock cycle, the WFSR transitions from state $\mathbf{S}_t = (\mathbf{s}_t, \mathbf{s}_{t+1}, \ldots, \mathbf{s}_{t+n-1})$ to the next state $\mathbf{S}_{t+1} = (\mathbf{s}_{t+1}, \mathbf{s}_{t+2}, \ldots, \mathbf{s}_{t+n})$ for $t \geq 0$, where the new word $\mathbf{s}_{t+n}$ is computed as

$$\mathbf{s}_{t+n} = F(\mathbf{s}_t, \mathbf{s}_{t+1}, \ldots, \mathbf{s}_{t+n-1}).$$

This iterative process produces a sequence of words $[\mathbf{s}] = \{\mathbf{s}_0, \mathbf{s}_1, \ldots, \mathbf{s}_n, \ldots\}$. The sequence $[\mathbf{s}]$ is said to be *ultimately periodic* if there exist integers $r \geq 1$ and $n_0 \geq 0$ such that $\mathbf{s}_{j+r} = \mathbf{s}_j$ for all $j \geq n_0$. If the periodicity begins from the start of the sequence (i.e., $n_0 = 0$), then the sequence is called *periodic*, and the smallest such r is referred to as the *period* of the sequence.

A WFSR is classified as *linear* if the feedback function F is linear; otherwise, it is referred to as a *nonlinear* WFSR. In case of a linear FSR of order n, the feedback function $F(\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{n-1}) = C_0\boldsymbol{x}_0 + C_1\boldsymbol{x}_1 + \ldots + C_{n-1}\boldsymbol{x}_{n-1}$ and then the corresponding feedback polynomial is $f(x) = C_0 + C_1x + \ldots + C_{n-1}x^{n-1} + x^n$ and its characteristic polynomial is $x^n f(1/x)$.

A WFSR (or its feedback function) is said to be *nonsingular* if every state has a unique predecessor, which implies that its state transition diagram is composed entirely of disjoint cycles. For a bit-oriented FSRs, it has been established in [8] that an FSR is nonsingular if and only if its feedback function takes the form

$$F(x_0, x_1, \ldots, x_{n-1}) = x_0 \oplus g(x_1, x_2, \ldots, x_{n-1}), \tag{2}$$

where the function $g(\cdot)$ is independent of the variable x_0.

A natural question that arises is whether an analogous necessary and sufficient condition exists for a WFSR to be nonsingular. It can be readily shown that a WFSR is nonsingular if its feedback function follows a structural form similar to that in Eq. (2). Further investigation shows that this condition is not sufficient. There are other WFSRs whose feedback functions are in different form. In [10], it is shown that an n-stage WFSR is nonsingular if and only if its feedback function $F \in \mathcal{MP}_m[\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{n-1}]$ can be represented as

$$F(\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{n-1}) = f_0(\boldsymbol{x}_0) \odot f_1(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{n-1}), \tag{3}$$

where f_0 is a bijective function and f_1 is an arbitrary function from $\mathbb{F}_{2^m}^{n-1} \to \mathbb{F}_{2^m}$. The operation $\odot$ is either $+$ or $\oplus$. Note that if $deg(F) = 1$, then the feedback function F satisfies Eq. (3) and therefore the feedback function is always nonsingular. In Table 1, some well-known FSRs are shown as the special case of WFSR by imposing different restrictions. Column 1 of Table 1 shows the name of the FSR. Columns 2 and 3 give the value of the degree of feedback function F and the word size m, respectively. Column 4 tells which operation is used for $\sum$ in Eq. (1) and Column 5 says, the multiplication operation used when $deg(F) > 1$.

FSR	$deg(F)$	m	$\sum$	$\prod$
LFSR	1	1	$\oplus$	-
NLFSR	> 1	1	$\oplus$	&
ALFG	1	> 1	$+$	-
MRMM	1	> 1	$\oplus$	-

Table 1: Special cases of WFSRs.

2.1 LFSR

A WFSR is referred to as a bit-oriented FSR when the word size $m = 1$. In this case, the $m \times m$ matrix coefficient C_k of Eq. (1) becomes a scalar $c_k \in \mathbb{F}_2$. If $deg(F) = 1$, then $F(x_0, \ldots, x_{n-1}) = \bigoplus_{k=0}^{n-1} c_k x_k$. This expression corresponds to the feedback function of a widely recognized FSR known as LFSR, which has been extensively studied in the literature [8, 11, 9]. LFSR is the simplest shift register where the output bit is a linear function (typically XOR) of its previous state bits. An LFSR with a primitive feedback polynomial produces a bitstream with the maximum possible period, provided the initial state is nonzero. These bitstreams also exhibit most of the desirable statistical properties and so making them suitable for various cryptographic applications.

However, LFSRs are inherently linear, and to eliminate this linearity, various nonlinear techniques have been proposed in the literature [9]. One such technique involves operating LFSRs in scrambler mode, where the feedback value is modified by an external input. This approach has been adopted in several modern stream ciphers, including Grain, Trivium, and ACORN [12, 13]. Further details are discussed in Section 3.

2.2 ALFG

Another prominent variant of the FSR is the Additive Lagged Fibonacci Generator (ALFG), which can be interpreted as a specific instance of WFSR, as categorized in Table 1. In this case, the degree of the feedback function is 1, and the word size $m > 1$. The addition operation $\sum$ corresponds to modular addition, and the feedback coefficients C_k belong to the set $\{I_m, 0\}$. Given that each C_k is either the identity matrix I_m or the zero matrix, the coefficients reduce to binary scalars $c_k \in \mathbb{F}_2$. As a result, the feedback function of the ALFG takes the form

$$F(\boldsymbol{x}_0, \ldots, \boldsymbol{x}_{n-1}) = c_0\boldsymbol{x}_0 + c_1\boldsymbol{x}_1 + \cdots + c_{n-1}\boldsymbol{x}_{n-1},$$

where n denotes the order of the ALFG. It is a well-established fact that such a sequence becomes periodic provided $c_0 \neq 0$ and the maximum period achieved by an ALFG of order n and word size m is equal to $(2^n - 1)2^{m-1}$. An ALFG that attains this maximum period is termed a *primitive ALFG*.

For good randomness properties, the feedback polynomial of an ALFG is typically chosen to be primitive. In the case of LFSRs, a primitive feedback polynomial guarantees a maximum period; this criterion alone is insufficient for ALFGs. Brent [3] showed that, in order to attain the maximal period, the feedback polynomial $f(x)$ must satisfy following two conditions:

$$\begin{aligned} f(x)^2 + f(-x)^2 &\not\equiv 2f(x^2) \pmod 8 \\ f(x)^2 + f(-x)^2 &\not\equiv 2(-1)^n f(-x^2) \pmod 8. \end{aligned} \tag{4}$$

Furthermore, Marsaglia et al. [14] using matrix-theory demonstrates that an ALFG reaches its maximum period with the initial state containing at least one odd value if and only if the transition matrix A, satisfies the following condition: $O(A) \pmod 2 = 2^n - 1,\ O(A) \pmod{2^2} = 2(2^n - 1),\ O(A) \pmod{2^3} = 4(2^n - 1)$.

2.3 MRMM

MRMM is another special case of a WFSR. If $\deg(F) = 1$ with word size $m > 1$, and $\oplus$ is used in place of $\sum$ in an n-stage WFSR, then the expression of F in Eq. (1) becomes

$$F(\mathbf{x}_0, \ldots, \mathbf{x}_{n-1}) = \bigoplus_{k=0}^{n-1} C_k \mathbf{x}_k,$$

where $C_0, C_1, \ldots, C_{n-1} \in \mathbb{M}_m(\mathbb{F}_2)$. This is the general expression for the feedback function of an MRMM [7, 15, 16]. We denote by MRMM(m, n) an n-stage MRMM over $\mathbb{F}_{2^m}$. An MRMM is also known as a σ-LFSR [17]; however, throughout this paper, we use the term MRMM. It is always possible for a periodic word sequence $[\mathbf{s}]$ to have a Linear Recurring Relation (LRR) among the elements as

$$\mathbf{s}_{i+n} = \sum_{k=0}^{n-1} C_k \mathbf{s}_{i+k} \quad i \geq 0. \tag{5}$$

It is possible to associate Eq. (5) with a matrix polynomial. It is expressed as $M(x) = I_m x^n - C_{n-1}x^{n-1} - \cdots - C_1 x - C_0$ with matrix coefficients and is called the *matrix polynomial* of the MRMM(m, n). Then, $\det(M(x))$ is a polynomial of degree mn over $\mathbb{F}_2$.

The following proposition [18, Proposition 4.2] gives some basic facts about the MRMMs.

Proposition 2.1. *For the sequence* $[\mathbf{s}]$ *generated by the MRMM(m, n),*

(i) $[\mathbf{s}]$ *is ultimately periodic and its period is no more than* $2^{mn} - 1$*;*

(ii) *if* C_0 *is nonsingular, then* $[\mathbf{s}]$ *is periodic. Conversely, if* $[\mathbf{s}]$ *is periodic whenever the initial state is of the form* $(b, 0, \ldots, 0)$*, where* $b \in \mathbb{F}_{2^m}$ *with* $b \neq 0$*, then* C_0 *is nonsingular.*

Let the word sequence [**s**] be generated by a primitive MRMM(m, n). For $j = 1, 2, \ldots, , m$, consider $[s^{(j)}] = \{s_0^{(j)}, s_1^{(j)}, \ldots, , \}$ be the bit sequence, where $s_k^{(j)}$ is the j^{th} least significant bit (lsb) of m-bit word $\mathbf{s}_k$. Then, $\mathbf{s}_k = (s_k^{(m)}, \ldots, , s_k^{(2)}, s_k^{(1)}) \in \mathbb{F}_2^m$, for $k = 0, 1, \ldots, ,$ and each $s_k{}^{(j)} \in \mathbb{F}_2$. Now, the following result is due to Niederreiter [7, Lemma-1].

Lemma 2.2. *Let* [**s**] *be an arbitrary recursive vector sequence with the characteristic polynomial* $g(x) \in F_2[x]$ *of degree* mn *and each vector is* m*-bit wide. Then, the bit sequence* $[s^{(j)}]$ *for* $1 \leq j \leq m$ *will be a linear recurring sequence in* $\mathbb{F}_2$ *with the same characteristic polynomial* $g(x)$*. So if* $g(x)$ *is primitive, then the word sequence* [$\boldsymbol{s}$] *and each bit sequence* $[s^{(j)}]$ *attain the maximal period i.e.,* $(2^{mn} - 1)$*.*

Each bit sequence $[s^{(j)}]$ has period $(2^{mn}-1)$ in the case of a primitive MRMM(m, n) and is a circularly shifted version of any other bit sequence $[s^{(k)}]$ for $1 \leq j, k \leq m$.

The following corollary follows from Lemma 2.2 and gives the component-wise linear complexity of the sequences generated by primitive MRMM.

Corollary 2.3. *Let*

$$\mathbf{s}_i = \left(s_i^{(m)}, \ldots, s_i^{(1)}\right) \in \mathbb{F}_2^m \simeq \mathbb{F}_{2^m} \quad i = 0, 1, \ldots,$$

be a sequence of words generated by a primitive MRMM of order n *over* $\mathbb{F}_{2^m}$*. Then for each* $1 \leq j \leq m$*, the linear complexity of the* j^{th} *coordinate sequence* $[s^{(j)}] = \{s_0^{(j)}, s_1^{(j)}, \ldots, , \}$ *over* $\mathbb{F}_2$ *is* mn*.*

In the following section, we study the system of LFSRs and some of its properties under some specific conditions.

3 System of LFSRs

In several stream cipher designs, systems of FSRs are commonly used. Many well-known candidates from major cryptographic competitions adopt this approach. For example, Grain-128 and Trivium, both finalists in the eSTREAM competition [13], utilize FSR-based structures. Similarly, ACORN, a finalist in the CAESAR competition [12], also employs a system of LFSRs. In the Grain-128 cipher, two FSRs are used: one is a primitive LFSR, while the other is a Nonlinear Feedback Shift Register (NFSR). In contrast, Trivium uses three FSRs, all of which are NFSRs. ACORN employs six FSRs, all implemented as LFSRs. Both ACORN and Trivium operate their FSRs in scrambler mode, meaning each feedback value is modified by an external input. However, in the case of Grain-128, the LFSR operates in free-running mode, where no external input is used to alter the feedback value. This design choice provides a guaranteed lower bound on the periodicity of the output sequence [19], whereas such a bound is absent in the designs of ACORN and Trivium.

An FSR in free running mode ensures large lower bound for period where as scrambler mode FSR introduces nonlinearity. However, both schemes provide most of the statistical properties. In this section, we investigate only system of LFSRs having one LFSR runs in free running mode and remaining others run in scrambler mode.

Consider a system of LFSRs comprising of k LFSRs as depicted in Fig. 1. Let $\mathrm{LFSR}_0, \mathrm{LFSR}_1, \ldots, \mathrm{LFSR}_{k-1}$ be those k LFSRs. In this system of LFSRs, LFSR_0 runs in free running mode and for $j > 0$, LFSR_j runs in scrambler mode, where the external bit z_j is used to modify the feedback value of LFSR_j. The external bit z_j is calculated by a Boolean function f, using some states of all previous LFSRs as input. The Boolean function f can be either linear or nonlinear. Depending upon, the nature of Boolean function f, we study following two configurations.

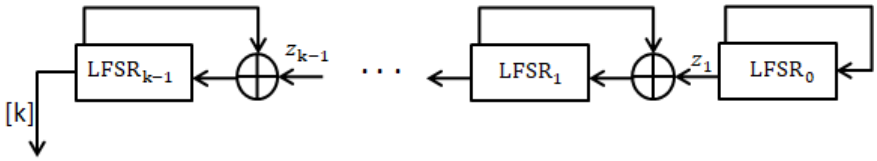


Figure 1: System of k LFSRs.

3.1 Linear Configuration

In this case, the external bit used to modify the feedback value of LFSR_j is a linear function of some of the states of all previous LFSRs. If $z_j = \mathrm{LFSR}_j[0]$, then it is famously known as the cascade connection of LFSRs [20]. Let P_j denotes the period of the LFSR_j, and assume that $\gcd(P_i, P_j) = 1$ for all $i \neq j$, with at least one nonzero initial state in LFSR_0. As established in [21], under these conditions, the period of the output bit sequence generated by the cascade connection is $P_1 P_2 \cdots P_k$. Moreover, the linear complexity of the resulting sequence is given by $\sum_{i=1}^{k} \log_2(P_i + 1)$.

When same primitive LFSR is used in the cascade system, then by [19], the period of the resulting system is a multiple by P. In this section, we demonstrate that the period of the cascade connection of two primitive LFSRs is always exactly $2P$. Furthermore, we determine the precise period of the cascade connection of k primitive LFSRs, all sharing the same feedback polynomial.

Let LFSR_i have order n_i with feedback polynomial $x^{n_i} - c_{i,n_i-1}x^{n-1} - \ldots - c_{i,0}$. Then the transition matrix of LFSR_i is denoted by

$$T_i = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_{i,0} & c_{i,1} & c_{i,2} & \cdots & c_{i,n_i-1} \end{pmatrix}_{n_i \times n_i}.$$

It can be verified that the matrix representing the cascade connection of k LFSRs, is given by:

$$\widetilde{A} = \begin{pmatrix} T_0 & & \dots & 0 \\ B_0 & T_1 & \dots & 0 \\ 0 & B_1 & T_2 & \dots \\ \vdots & \vdots & \vdots & \vdots \\ 0 & \dots & B_{k-1} & T_{k-1} \end{pmatrix}, \tag{6}$$

where

$$B_i = \begin{pmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 0 & \dots & 0 \end{pmatrix}_{n_{i+1} \times n_i}.$$

From Eq. (6), it follows that the characteristic polynomial of $\widetilde{A}$ is the product of the characteristic polynomials of the individual matrices T_i.

We note that $B_{i+1}B_i = \mathbf{0}$ for every i. Hence, it can be verified that the $(i,j)^{th}$ block entry of $\widetilde{A}^s$ is given by

$$\left(\widetilde{A}^s\right)_{i,j} = \begin{cases} 0, & i < j, \\ T_{i-1}^s, & i = j, \\ \displaystyle\sum_{\substack{l_0,l_1,\dots,l_t \geq 0 \\ l_0+l_1+\dots+l_t = s-t}} T_{i-1}^{l_0} B_{i-2} T_{i-2}^{l_1} \cdots B_{j-1} T_{j-1}^{l_t}, & i > j,\ s \geq t, \\ 0, & i > j,\ s < t, \end{cases} \tag{7}$$

where $t = i - j$.
Hence,

$$\widetilde{A}^s = \begin{pmatrix} T_0^s & 0 & 0 & \dots & 0 \\ \displaystyle\sum_{l_0+l_1=s-1} T_1^{l_0} B_0 T_0^{l_1} & T_1^s & 0 & \dots & 0 \\ \displaystyle\sum_{l_0+l_1+l_2=s-2} T_2^{l_0} B_1 T_1^{l_1} B_0 T_0^{l_2} & \displaystyle\sum_{l_0+l_1=s-1} T_2^{l_0} B_1 T_1^{l_1} & T_2^s & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \displaystyle\sum_{\sum_{r=0}^{k-1} l_r = s-(k-1)} T_{k-1}^{l_0} B_{k-2} T_{k-2}^{l_1} \dots B_0 T_0^{l_{k-1}} & \displaystyle\sum_{\sum_{r=0}^{k-2} l_r = s-(k-2)} T_{k-1}^{l_0} B_{k-2} T_{k-2}^{l_1} \dots B_1 T_1^{l_{k-2}} & \dots & \dots & T_{k-1}^s \end{pmatrix},$$

with the convention that the summation is defined to be zero whenever $s < t$, that is,

$$\sum_{\sum_{r=0}^{t} l_r = s-t} T_{i-1}^{l_0} B_{i-2} T_{i-2}^{l_1} \dots B_{j-1} T_{j-1}^{l_t} = 0 \quad \text{if } s < t.$$

If all LFSRs are same with order n, then $B_1 = B_2 = \dots = B_{k-1} = B$. This structural uniformity simplifies the analysis of the composite transition matrix $\widetilde{A}$ defined in (6), and will be crucial in studying the behavior of powers of $\widetilde{A}$ and determining the period of the cascade system.

Lemma 3.1. *Let $f(x) = a_0 + a_1x + \ldots + a_{n-1}x^{n-1} + x^n$ be a primitive characteristic polynomial of an LFSR with transition matrix T. Then, for the cascade matrix $\widetilde{A}$ defined for k identical LFSRs, we have*

$$f(\widetilde{A})^i \neq \mathbf{0} \quad \textit{for all } i < k.$$

Proof. It is enough to show that $f(\widetilde{A})^{k-1} \neq \mathbf{0}$. Now,

$$f(\widetilde{A})^{k-1} = \left(a_0 I + a_1\widetilde{A} + \ldots + \widetilde{A}^n\right)^{k-1} = b_0 I + b_1\widetilde{A} + \ldots + \widetilde{A}^{n(k-1)},$$

where the b_i's are suitable linear combinations of the a_j's.
Consider the $(k,1)^{th}$ block of $f(\widetilde{A})^{k-1}$, which is given by

$$\sum_{j=1}^{(k-1)n} b_j \sum_{\ell_0+\ell_1+\ldots+\ell_{k-1}=j-(k-1)} T^{\ell_0}B\cdots BT^{\ell_{k-1}}.$$

Let $V = (0,0,\ldots,1)^T$. For $\ell_{k-1} < n-1$, we have

$$T^{\ell_{k-1}}V = (0,0,\ldots,1,*,\ldots,*)^T,$$

so

$$BT^{\ell_{k-1}}V = (0,0,\ldots,0)^T,$$

and thus

$$T^{\ell_0}B\cdots BT^{\ell_{k-1}}V = (0,0,\ldots,0)^T.$$

Now suppose $\ell_{k-1} \geq n-1$ and $\ell_{k-2} < n-1$. Then again,

$$BT^{\ell_{k-1}}V = (0,0,\ldots,0)^T \text{ or } (0,0,\ldots,1)^T,$$

but in either case,

$$BT^{\ell_{k-2}}BT^{\ell_{k-1}}V = (0,0,\ldots,0)^T.$$

Continuing this way, we conclude that for the product $T^{\ell_0}B\cdots BT^{\ell_{k-1}}V$ to be non-zero, we must have $\ell_1, \ell_2, \ldots, \ell_{k-1} \geq n-1$. Given the constraint $\ell_0 + \ldots + \ell_{k-1} = j-k+1 \leq (k-1)n-k+1$, this forces $\ell_0 = 0$ and $\ell_1 = \ell_2 = \ldots = \ell_{k-1} = n-1$. Therefore,

$$T^{\ell_0}BT^{\ell_1}\cdots BT^{\ell_{k-1}}V = BT^{n-1}\cdots BT^{n-1}V = (0,0,\ldots,1)^T.$$

This implies that in the sum $\sum_{j=1}^{(k-1)n} b_j \sum_{\ell_0+\cdots+\ell_{k-1}=j-k+1} T^{\ell_0}B\cdots BT^{\ell_{k-1}}V$, only non vanishing term is

$$b_{(k-1)n}T^{n-1}BT^{n-1}\ldots BT^{n-1}V,$$

which is equal to V. Hence

$$\sum_{j=1}^{(k-1)n} b_j \sum_{\ell_0+\ell_1+\ldots+\ell_{k-1}=j-(k-1)} T^{\ell_0} B \cdots B T^{\ell_{k-1}} V = V,$$

which tells 1 is the eigen value of the matrix on the LHS.
So the first entry of the last block row of $f(\widetilde{A})^{k-1}$ is non-zero. Hence, $f(\widetilde{A})^{k-1} \neq \mathbf{0}$, completing the proof. □

Theorem 3.2. *Let $\widetilde{A}$ be the matrix representation of a cascade of k identical primitive LFSRs. Then, the period of $\widetilde{A}$ is $2^t P$, where P is the period of the LFSR and t is the smallest positive integer such that $2^t \geq k$.*

Proof. Let $f(x) = a_0 + a_1 x + \ldots + a_{n-1}x^{n-1} + x^n$ be the characteristic polynomial of T, and suppose $O(T) = P$. Then the characteristic polynomial of $\widetilde{A}$ is $f(x)^k$. Since $f(x)$ is primitive, the minimal polynomial of $\widetilde{A}$ must be one among $1, f(x), f(x)^2, \ldots, f(x)^k$. By Lemma 3.1, we have $f(\widetilde{A})^i \neq \mathbf{0}$ for every $i \leq k-1$. Therefore, the minimal polynomial of $\widetilde{A}$ is exactly $f(x)^k$. It then follows that $O(\widetilde{A}) = O(f(x)^k) = 2^t p$, where t satisfies the divisibility condition stated in Theorem 3.8 of [11]. □

In the preceding theorem, the characteristic polynomials are identical. A natural question that arises is: What is the periodicity of the cascade connection when the characteristic polynomials of all individual LFSRs are distinct? This question was partially answered in [21] when the GCD of the periods is relatively prime. The next theorem answers this question completely.

Theorem 3.3. *Let $\widetilde{A}$ be the matrix representing a cascade connection of k primitive LFSRs, with respective periods $P_1, P_2, \ldots, P_k$ and distinct characteristic polynomials. Then, the period of the matrix $\widetilde{A}$ is given by*

$$\mathrm{lcm}(P_1, P_2, \ldots, P_k).$$

Proof. Since each LFSR is primitive, the characteristic polynomials
$f_1(x), f_2(x), \ldots, f_k(x)$ are primitive and hence irreducible over $\mathbb{F}_2$. As these polynomials are distinct, they are pairwise coprime. Thus, the characteristic polynomial of the overall system matrix $\widetilde{A}$ is

$$F(x) = \prod_{i=1}^{k} f_i(x).$$

Using an argument similar to that of Lemma 3.1, one can show that for each j, the product

$$F_j(\widetilde{A}) = \prod_{i \neq j} f_i(\widetilde{A}) \neq 0,$$

which implies that no proper divisor of $F(x)$ annihilates $\widetilde{A}$. Therefore, the minimal polynomial of $\widetilde{A}$ is $F(x)$. By Theorem 3.9 in [11], the order of $\widetilde{A}$, denoted $O(\widetilde{A})$, is given by

$$O(\widetilde{A}) = O(F(x)) = \mathrm{lcm}(O(f_1(x)), O(f_2(x)), \ldots, O(f_k(x))) = \mathrm{lcm}(P_1, P_2, \ldots, P_k).$$

□

3.2 Nonlinear Configuration

In this section, we examine a system of primitive LFSRs, all sharing a common feedback polynomial that satisfies Eq. (4), where the external bit is generated through a non-linear function of some states of all preceding LFSRs. Before that, we first analyze a system consisting of two LFSRs, where the nonlinear function is defined as the carry bit from the feedback value of LFSR_0. This carry bit serves as the external bit and is used to modify the feedback value of LFSR_1.

Let LFSR_0 and LFSR_1 generate the sequences $[x]$ and $[y]$, respectively, in free-running mode. The carry-bit sequence produced by LFSR_0 which is used to modify the feedback value of LFSR_1 is denoted by $[z]$, while $[k]$ represents the output sequence generated by the combined system of the two LFSRs. Denote the corresponding periods of $[x]$, $[y]$, and $[z]$ by P_x, P_y, and P_z. Now, we have the following results.

Lemma 3.4. *Let A be the transition matrix of a primitive LFSR with period $2^n - 1$. Then, for any $1 \leq t < 2^n - 1$, the matrix $A^t - I$ is invertible.*

Proof. Suppose $A^t - I$ is not invertible for some $1 \leq t < 2^n - 1$. Then there exists a nonzero vector V such that $A^t V = V$, implying that the sequence $V, AV, A^2V, \ldots$ is periodic with period dividing t. This contradicts the fact that $O(A) = 2^n - 1$ Hence, $A^t - I$ is invertible. □

Lemma 3.5. *The period of the carry-bit sequence divides the period of $LFSR_0$.*

Proof. Let P_z and P_x denote the periods of the carry-bit sequence $[z]$ and the state sequence $[x]$, respectively. Since $[z]$ is determined by $[x]$, we have $P_z \leq P_x$.

Suppose, for contradiction, that $P_z \nmid P_x$. Then $P_x = kP_z + r$ for some integer k and $0 < r < P_z$. Since P_x is the period of $[x]$, it follows that $x_{n+P_x} = x_n$, and hence $z_{n+P_x} = z_n$. Thus,

$$z_{n+r} = z_{n+kP_z+r} = z_{n+P_x} = z_n,$$

which implies that r is a period of $[z]$, contradicting the minimality of P_z. Therefore, $P_z \mid P_x$. □

Theorem 3.6. *The period of the carry-bit sequence generated by the primitive $LFSR_0$ is $2^n - 1$.*

Proof. Let $f(x) = x^n + c_{n-1}x^{n-1} + \cdots + c_1x + c_0$ be the feedback polynomial of LFSR_0. Since A is the transition matrix of the primitive LFSR whose feedback polynomial satisfies Eq. (4), $O(A) = 2(2^n - 1)$ in $\mathbb{Z}_4$ [3]. Denote the initial state by $X_0 = (x_0, x_1, \ldots, x_{n-1})^T$, where $X_0 \neq 0$. By Lemma 3.5, the period of the carry-bit sequence $[z]$ satisfies $P_z = \frac{2^n-1}{k}$ for some positive integer k.

Suppose, for contradiction, that $k \neq 1$, so that $P_z = u = \frac{2^n-1}{k} < 2^n - 1$. To analyze the period of the carry bits, we lift the recurrence relation to $\mathbb{Z}_4$ to express the feedback relation together with the carry terms in a unified algebraic form. The feedback relation in $\mathbb{Z}_4$ is given by

$$x_n + 2z_0 = c_0x_0 + c_1x_1 + \cdots + c_{n-1}x_{n-1},$$

which gives

$$x_n = c_0x_0 + c_1x_1 + \cdots + c_{n-1}x_{n-1} - 2z_0,$$

where z_0 is the first carry bit. Consequently, the next state of the sequence in $\mathbb{Z}_4$ satisfies

$$X_1 = AX_0 - Wz_0,$$

where $W = (0, 0, \ldots, 0, 2)^T \in \mathbb{Z}_4^n$.

To illustrate the recursive structure, we compute the next few states explicitly. The second state is

$$X_1 = AX_0 - Wz_0.$$

The third state is

$$X_2 = AX_1 - Wz_1 = A^2X_0 - AWz_0 - Wz_1,$$

where z_1 is the carry bit corresponding to x_{n+1}. Continuing in this way, for $2 \leq i \leq u + 1$, the i^{th} state satisfies

$$X_{i-1} = A^{i-1}X_0 - \sum_{j=0}^{i-2} A^{i-j-2}Wz_j.$$

Since the carry-bit sequence $[z]$ has period u, the state at step $u + 1$ satisfies

$$X_u = A^uX_0 - (A^{u-1} + I)Wz_0 - \sum_{j=1}^{u-1} A^{u-j-1}Wz_j.$$

By continuing this process, for $u + 2 \leq i \leq 2u + 1$, the i^{th} state satisfies

$$X_{i-1} = A^{i-1}X_0 - \sum_{j=0}^{u-1}(A^{i-j-2} + A^{i-u-j-2})Wz_j,$$

with the convention that $A^t = 0$ for $t < 0$.

Now consider the state at step $ku + 2$. Using the recursive expression derived above, we have

$$X_{ku+1} = A^{ku+1}X_0 - \sum_{j=0}^{u-1} \left(A^{ku-j} + A^{ku-u-j} + \cdots + A^{u-j} + A^{-j}\right) Wz_j.$$

We know that

$$A^{ku-u+1} + A^{ku-2u+1} + \cdots + A = A(A^u - I)^{-1}(A^{uk} - I),$$

where the inverse exists because $A^u - I$ is invertible over $\mathbb{F}_2$ by Lemma 3.4, and invertibility lifts to $\mathbb{Z}_4$.

Since A is the transition matrix of a primitive polynomial of degree n, it satisfies $A^{2^n-1} = I$ over $\mathbb{F}_2$. Thus, $A^{uk} - I = 0$ in $\mathbb{F}_2$, implying that $A^{uk} - I$ has entries either 0 or 2 in $\mathbb{Z}_4$. Consequently,

$$(A^{ku-u+1} + A^{ku-2u+1} + \cdots + A)W = 0 \quad \text{in } \mathbb{Z}_4.$$

Therefore, the expression for X_{ku+1} reduces to

$$X_{ku+1} = A^{ku+1}X_0 - \sum_{j=0}^{u-1} Q_j W z_j,$$

where $Q_j = A^{ku-j} + A^{ku-u-j} + \cdots + A^{u-j} + A^{-j}$.
We observe that the coefficient of z_j in the above expression is Q_jW, and for each $j \geq 1$, $Q_jW = 0$ in $\mathbb{Z}_4$ as Q_j is obtained by multiplying A^{u-j-1} to $A^{ku-u+1} + A^{ku-2u+1} + \ldots + A$. Therefore, the entire sum involving the carry bits vanishes for $j \geq 1$, and we obtain

$$X_{ku+1} = A^{ku+1}X_0 - Q_0 W z_0.$$

Since period of X_i is $2^n - 1$,$X_{ku+1} = X_1$ and $Q_0W = (A^{ku-u+1} + A^{ku-2u+1} + \cdots + A + I)W = IW$,

$$A^{ku}X_0 = X_0.$$

which is the contradiction to the fact that $O(A) = 2(2^n - 1)$ in $\mathbb{Z}_4$

Therefore, $k = 1$, and the period of the carry-bit sequence is $2^n - 1$. □

Theorem 3.7. *Consider the system of 2 LFSRs as described above. Then the period of* $[k]$ *is* $2(2^n - 1)$.

Proof. Let $Y_0 = (y_0, \ldots, y_{n-1})$ be the initial state of LFSR_2. The feedback computation in free-running mode satisfies

$$y_{n+i} = c_0 y_i \oplus c_1 y_{i+1} \oplus \cdots \oplus c_{n-1} y_{n+i-1}.$$

Additionally, the carry bit z_i from LFSR_1 is XORed with the $(n + i)^{th}$ state of LFSR_2. Consequently, the sequence evolves as

$$y_0 = k_0,\ y_1 = k_1,\ \ldots,\ y_{n-1} = k_{n-1},\ k_n,\ k_{n+1},\ \ldots,$$

with $k_n = y_n \oplus z_0$ and $k_{n+1} = y_{n+1} \oplus c_{n-1}z_0 \oplus z_1$. Since $\{k_i\}$ is generated from $\{y_i\}$ by XORing with carry terms, $\{k_i\}$ satisfies the recurrence relation

$$k_{n+i} = c_0k_i \oplus c_1k_{i+1} \oplus \cdots \oplus c_{n-1}k_{i+n-1} \oplus z_i \quad \text{for all } i \geq 0.$$

To describe the contribution of the carry bits, let A be the transition matrix corresponding to the primitive characteristic polynomial

$$f(x) = x^n + c_{n-1}x^{n-1} + \cdots + c_0,$$

and define $v = (0, 0, \ldots, 0, 1)^T$. For $0 \leq i \leq 2^n - 2$, the coefficient of z_0 in k_{n+i} is the $(n, 1)^{th}$ entry of A^iv, denoted by l_i. Since $f(x)$ is primitive, $\{l_i\}$ is periodic with period $2^n - 1$. Further, for $2 \leq i \leq n - 1$,

$$l_{2^n-i} = [A^{2^n-i}v]_n = [A^{-(i-1)}v]_n = 0.$$

Thus, the feedback computation of LFSR_2 in scrambler mode can be written as

$$k_{n+i} = y_{n+i} \oplus l_iz_0 \oplus l_{i-1}z_1 \oplus \cdots \oplus l_0z_i \quad \text{for } 0 \leq i \leq 2^n - 2.$$

Since the period of $[z]$ is $2^n - 1$, for $i = 2^n - 1$

$$k_{n+2^n-1} = y_{n+2^n-1} \oplus z_0 \oplus l_{2^n-2}z_1 \oplus \cdots \oplus l_0z_{2^n-2} \oplus z_0.$$

Therefore, the coefficient of z_0 in k_{n+2^n-1} is 0. Since the coefficients of z_j in k_{n+i} are inherited recursively from z_{i-1} in k_{n+i-1}, it follows that the coefficients of z_j are 0 for $0 \leq j \leq 2^n - n - 1$ in k_{n+i} for $i \geq 2^n - 1 + j$. Finally, we obtain

$$k_{2(2^n-1)} = y_{2(2^n-1)} \oplus l_{2^n+n-2}z_{2^n-n} \oplus l_{2^n+n-3}z_{2^n-n+1} \oplus \cdots \oplus l_{2^n-2}z_{2^n-2}.$$

Since $l_{2^n-i} = 0$ for $2 \leq i \leq n - 1$, all the carry bit terms vanish, and hence

$$k_{2(2^n-1)} = y_{2(2^n-1)}.$$

As $\{y_i\}$ has period $2^n - 1$, it follows that the period of $\{k_i\}$ is $2(2^n - 1)$. □

Having analyzed the system having two LFSRs and established that its period is $2(2^n - 1)$, we now extend this construction to a system involving k LFSRs.

In this configuration, the feedback value of LFSR_i is modified by a nonlinear function derived from the carry bits of the preceding LFSRs $\text{LFSR}_0, \ldots, \text{LFSR}_{i-1}$. Specifically, this function is defined as the XOR of the carry bits obtained from the feedback computations of $\text{LFSR}_{i-1}, \text{LFSR}_{i-2}, \ldots, \text{LFSR}_0$. We show that a system of LFSRs configured in this manner, where all LFSRs share the same feedback polynomial, is functionally equivalent to an ALFG. As discussed in [4], an ALFG of order n can be viewed as a scrambler composed of m binary LFSRs, where the feedback polynomial is typically a trinomial of the form:

$$f(x) = x^n + c_{n-1}x^{n-1} + \cdots + c_1x + c_0.$$

In their study, the feedback computation in ALFG was limited to two tap points, resulting in the carry bit influencing only the next LFSR in sequence. However, when the feedback polynomial includes more than two tap positions, the carry bit propagates across multiple subsequent LFSRs. In this work, we consider the general case where the number of tap points exceeds two. We present a formal mathematical proof demonstrating that an ALFG with such a feedback structure can be equivalently represented using the system of LFSRs described below.

Theorem 3.8. *Consider an ALFG of order n and word size m with feedback polynomial $f(x) = x^n + c_{n-1}x^{n-1} + \ldots + c_1x + c_0$. Then, ALFG can be expressed in terms of m LFSRs having all LFSRs have same feedback polynomial $f(x)$.*

Proof. Let $\mathbf{S}_0 = (\boldsymbol{l}_0, \boldsymbol{l}_1, \ldots, \boldsymbol{l}_{n-1})$ be the initial state of ALFG, where each $\boldsymbol{l}_i$ is an m-bit integer for $0 \leq i < n$. Now, define $\mathrm{s}_{i,j} = (\boldsymbol{l}_i >> j)\&1$, for $0 \leq i < n$ and $0 \leq j < m$, i.e., $\mathrm{s}_{i,j}$ is the $(j+1)^{th}$ lsb bit of $\boldsymbol{l}_i$. Next, consider m LFSRs, denoted as $\mathrm{LFSR}_0, \ldots, \mathrm{LFSR}_{m-1}$, where LFSR_k is initialized with the bit sequence $\mathbf{s}_k = (s_{0,k}, s_{1,k}, \ldots, s_{n-1,k})$. Let $\mathrm{L}_k[0], \ldots, \mathrm{L}_k[n-1]$ be the states of LFSR_k, then $\mathrm{L}_k[j] = s_{j,k}$. After initializing all the LFSRs, they are updated according to their respective feedback rules and the states of ALFG is reconstructed from the updated states of the LFSRs as follows:

i. **For $\mathbf{LFSR}_0$**: As LFSR_0 runs in free running mode, execute the following steps:

- Compute the feedback sum $\mathrm{FB}_0 = \sum_i c_i \mathrm{L}_0[i]$.
- Shift of states: $\mathrm{L}_0[i] = \mathrm{L}_0[i+1]$, for $0 \leq i < n$.
- Update $\mathrm{L}_0[n-1] = \mathrm{FB}_0\&1$.
- Compute Carry number $\mathrm{C}_0 = \mathrm{FB}_0/2$. Note that the size of C_0 is atmost $log_2(t)$, where t is the number of tap points in the feedback function. The bits of C_0 represent the carry bits from LFSR_0 to other LFSRs. The first lsb of C_0 is the carry bit from LFSR_0 to LFSR_1, denoted as $z_{0,1}$. The 2nd lsb of C_0 is the carry bit from LFSR_0 to LFSR_2, denoted as $z_{0,2}$ and so on.

ii. **For $\mathbf{LFSR}_k$**, for $1 \leq k < m$:

- Compute the feedback sum $\mathrm{FB}_k = \sum_i c_i \mathrm{L}_k[i] + \mathrm{C}_{k-1}$.
- Shift of states: $\mathrm{L}_k[i] = \mathrm{L}_k[i+1]$, for $0 \leq i < n$.
- Update $\mathrm{L}_k[n-1] = \mathrm{FB}_k\&1$.
- Compute Carry number $\mathrm{C}_k = \mathrm{FB}_k/2$.

It is clear that each element $\boldsymbol{l}_k$ of the ALFG state can be reconstructed from the states of the m LFSRs as $\boldsymbol{l}_k = \mathrm{L}_{m-1}[k] \parallel \mathrm{L}_{m-2}[k] \parallel \ldots \parallel \mathrm{L}_0[k]$, for $0 \leq k < m$. Now, it remains to show that the feedback value of the ALFG is given by $\mathrm{L}_{m-1}[n-$

1] $\|$ $L_{m-2}[n-1] \| \ldots \| L_0[n-1]$. This is trivial, as the feedback value of ALFG is the realization of binary addition of t many $m-$bit numbers. This completes the proof. □

In the following example, an ALFG is described in terms of LFSRs.

Rnd No.	L_0[0 to 4]							L_1[0 to 4]							L_2[0 to 4]							L_3[0 to 4]							ALFG [0 to 4]				
	0	1	2	3	4	FB_0	FS_0	0	1	2	3	4	FB_1	FS_1	0	1	2	3	4	FB_2	FS_2	0	1	2	3	4	FB_3	FS_3	l_0	l_1	l_2	l_3	l_4
1:	1	1	0	0	0	0	2	1	0	0	0	0	0	2	0	1	0	0	0	0	2	0	0	0	0	0	1	1	03	05	00	00	00
2:	1	0	0	0	0	1	1	0	0	0	0	0	0	0	1	0	0	0	0	1	1	0	0	0	0	1	1	1	05	00	00	00	08
3:	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	1	1	1	1	00	00	00	08	0D
4:	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	0	1	1	1	0	2	00	00	08	0D	0D
5:	0	0	1	1	1	0	2	0	0	0	0	0	1	1	0	0	1	1	1	0	2	0	1	1	1	0	1	3	00	08	0D	0D	05
6:	0	1	1	1	0	0	2	0	0	0	0	1	0	2	0	1	1	1	0	1	3	1	1	1	0	1	1	5	08	0D	0D	05	0A
7:	1	1	1	0	0	1	3	0	0	0	1	0	1	1	1	1	1	0	1	0	4	1	1	0	1	1	1	3	0D	0D	05	0A	0C
8:	1	1	0	0	1	1	3	0	0	1	0	1	1	3	1	1	0	1	0	1	3	1	0	1	1	1	0	4	0D	05	0A	0C	0B
9:	1	0	0	1	1	0	2	0	1	0	1	1	1	3	1	0	1	0	1	0	4	0	1	1	1	0	0	2	05	0A	0C	0B	07
10:	0	0	1	1	1	0	2	1	0	1	1	0	1	3	0	1	0	1	0	0	2	1	1	1	0	0	0	4	0A	0C	0B	07	02

Table 2: An example of ALFG in terms of LFSRs.

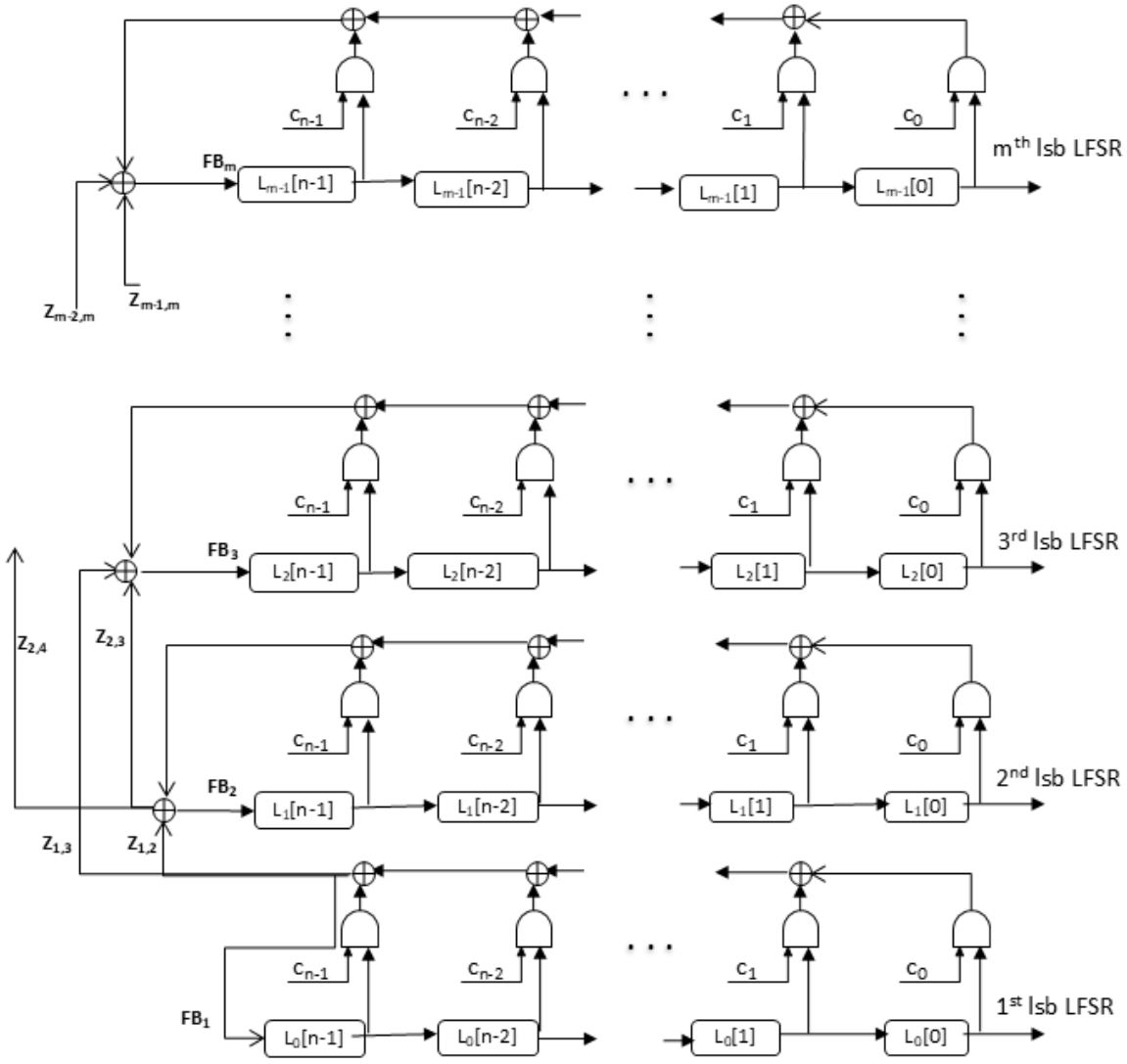
Example 1. Consider an ALFG of order 5 with word size 4 having feedback polynomial $x^5 + x^4 + x^2 + x + 1$. If $\boldsymbol{l}_0, \boldsymbol{l}_1, \ldots, \boldsymbol{l}_4$ are the states of ALFG with initial states as $\boldsymbol{l}_0 = 3$, $\boldsymbol{l}_1 = 5$, $\boldsymbol{l}_2 = \boldsymbol{l}_3 = \boldsymbol{l}_4 = 0$. The column 1 in Table 2 tells the round number where as the columns 2 to 6 tell about the values of the states of $LFSR_0$ (i.e., $\mathbf{L}_0[0], \ldots, \mathbf{L}_0[4]$) with respect to the iteration number. The column named as F_k, FS_k and C_k tell the feedback-bit value, feedback sum and carry sum of $LFSR_k$ respectively. Since the word size m is 4 in this example, the total number of LFSRs to represent ALFG is 4. Using $\boldsymbol{l}_k$, initialize $L_0[k], L_1[k], \ldots, L_3[k]$, for $0 \leq k < 5$. Once four LFSRs are initialized, the states each four LFSRs are updated and the states of ALFG is calculated from the states of these LFSRs as shown in Table 2. Thus, states of ALFG can derived using the states of LFSRs.

Figure 2 represents the pictorial description of n^{th} order ALFG in terms of LFSRs.

4 MRMM based cascade systems

In Section 3.1, the cascade system of LFSRs is studied. Now we investigate the properties of word-oriented cascade system of MRMMs. Consider k MRMMs, where ith MRMM is denoted by $MRMM_i(m_i, n_i)$. The associated matrix polynomial in $\mathbb{M}_m(\mathbb{F}_2)[x]$ is given by:

$$M_i(x) = x^{n_i} - C_{i,n_i-1}x^{n_i-1} - \cdots - C_{i,1}x - C_{i,0}$$



FB_k = Feedback Value for k^{th} lsb LFSR
$z_{j,k}$ = Carry-bit from j^{th} lsb LFSR to k^{th} lsb LFSR
$\mathrm{L}_j[i] = (i+1)^{th}$ state of j^{th} lsb LFSR
$c_k = k^{th}$ coefficient of the feedback function

Figure 2: ALFG of n^{th} order in terms of LFSRs.

where the coefficient matrix $C_{i,j}$ has dimension $m_i \times m_i$, for every $1 \leq i \leq k$. It is always possible to associate $M_i(x)$ with an (m_i, n_i)-block companion matrix

$T_i \in M_{m_i n_i}(\mathbb{F}_2)$ in the following form

$$(T_i)_{m_i n_i \times m_i n_i} = \begin{pmatrix} 0 & I_{m_i} & 0 & \cdots & 0 \\ 0 & 0 & I_{m_i} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & I_{m_i} \\ C_{i,0} & C_{i,1} & C_{i,2} & \cdots & C_{i,n_i-1} \end{pmatrix}_{n_i \times n_i},$$

where $\mathbf{0}$ indicates the zero matrix in $\mathbb{M}_{m_i}(\mathbb{F}_2)$, while I_{m_i} denotes the $m_i \times m_i$ identity matrix over $\mathbb{F}_2$. It is easy to see that the characteristic polynomial of the $\mathrm{MRMM}_i(m_i, n_i)$ is same as the characteristic polynomial of $T_i = \det(T_i - xI)$.

It was shown in [21] that the cascade connection of two MRMMs is represented by the block matrix:

$$\widetilde{T} = \begin{pmatrix} T_1 & 0 \\ B & T_2 \end{pmatrix},$$

where the matrix B has the form:

$$B = \begin{pmatrix} \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{\mathbf{I}} & \mathbf{0} & \dots & \mathbf{0} \end{pmatrix}_{m_2 n_2 \times m_1 n_1}.$$

Each block entry in the matrix B has dimensions $m_2 \times m_1$, and $\tilde{\mathbf{I}}$ is a diagonal matrix with $\min(m_1, m_2)$ ones along its diagonal and all other entries zero. Let the periods of the MRMM_1 and MRMM_2 systems be P_1 and P_2, respectively. As established in [21, Theorem 1], we have that P_1 divides P_2. Furthermore, [21, Theorem 10] shows that the period of the combined transformation $\widetilde{T}$ is $P_1 P_2$ when $\gcd(P_1, P_2) = 1$.

In the special case of a system consisting of two LFSRs with identical feedback polynomials, Theorem 3.2 demonstrates that the period of $\widetilde{T}$ is $2P$, where P is the period of each individual LFSR. A natural question arises: what happens when the LFSRs are replaced by MRMMs? The following result provides a partial answer to this question.

Theorem 4.1. *For a cascade connection of two primitive MRMMs having the same matrix polynomial with period P, the period of the matrix $\widetilde{T}$ divides $2P$.*

Proof. It can be verified that

$$\widetilde{T}^{2P+1} = \begin{pmatrix} T_1 & 0 \\ \sum_{k=0}^{2P} T_1^k B T_1^{2P-k} & T_1 \end{pmatrix}.$$

But,

$$\begin{aligned} \sum_{k=0}^{2P} T_1^k B T_1^{2P-k} &= B T_1^{2P} + T_1 B T_1^{2P-1} + \ldots + T_1^{P-1} B T_1^{P+1} + T_1^P B T_1^P \\ &+ T_1^{P+1} B T_1^{P-1} + \ldots + T_1^{2P-1} B T_1 + T_1^{2P} B \\ &= B + T_1 B T_1^{P-1} + \ldots + T_1^{P-1} B T_1 + B \\ &+ T_1 B T_1^{P-1} + \ldots + T_1^{P-1} B T_1 + B \\ &= B. \end{aligned}$$

Therefore, $\widetilde{T}^{2P+1} = \begin{pmatrix} T_1 & 0 \\ B & T_1 \end{pmatrix} = \widetilde{T}$. As $\widetilde{T}$ is invertible, $\widetilde{T}^{2P} = I$ and so the period of $\widetilde{T}$ must divide $2P$. □

In case of primitive MRMM$(m, 2n)$, the period of the word sequence generated by it is $2^{2mn} - 1$ and the linear complexity is $2mn$. However, for a cascade system comprising two such MRMMs, the period can reach $(2^{mn} - 1)^2$, with a significantly higher linear complexity of m^2n^2. Although the periods in both cases are comparable, the cascade system exhibits superior linear complexity. Moreover, the cascade system demonstrates a faster avalanche effect compared to a single MRMM. Therefore, cascading two MRMMs results in improved randomness properties. In the following section, we experimentally analyze the randomness properties of bitstreams generated by three different MRMM-based cascade systems.

4.1 Avalanche Property

In this section, we analyze the avalanche effect on the internal states of three MRMM-based cascade systems named as $\mathbf{CS}_1, \mathbf{CS}_2$ and $\mathbf{CS}_3$. We cannot have a formal mathematical proof of our study on avalanche property, however we provide an experimental results on it. In our experimental setup, the used three cascade systems $\mathbf{CS}_1, \mathbf{CS}_2$ and $\mathbf{CS}_3$ are depicted in Fig. 3, Fig. 4 and Fig. 5, respectively. To analyze the avalanche effect in cascade systems, we begin by initializing its internal state. We then experimentally determine the number of iterations required for the system to exhibit the avalanche property, which we define as a 50% change in the bits of the internal state relative to the initial state. Our experiments reveal that the number of iterations needed to reach this 50% avalanche threshold depends on the Hamming weight of the initial state. Specifically, initial states with higher Hamming weight tend to require fewer iterations to achieve the avalanche effect.

In contrast, when the initial state is randomly balanced (i.e., has an approximately equal number of ones and zeros), the number of iterations required remains relatively consistent across trials. It is observed that these results are consistent across different values of m and n. In this paper, we present results for three configurations of **CS**, each consisting of 12 registers, where each register has a bit size of 16.

In this setup, $\mathbf{CS}_k$ denotes a cascade system consisting of k MRMMs, where one MRMM operates in free-running mode, and the remaining $k-1$ MRMMs operate in scrambler mode.

- $\mathbf{CS}_1$ includes a single MRMM, denoted by $\mathrm{CS}_{1,0}$, which runs in free-running mode. The order of $\mathrm{CS}_{1,0}$ is 12.
- $\mathbf{CS}_2$ consists of two MRMMs: $\mathrm{CS}_{2,0}$ and $\mathrm{CS}_{2,1}$, each of order 6. Here, $\mathrm{CS}_{2,0}$ runs in free-running mode, while $\mathrm{CS}_{2,1}$ operates in scrambler mode.
- $\mathbf{CS}_3$ contains three MRMMs: $\mathrm{CS}_{3,0}$, $\mathrm{CS}_{3,1}$, and $\mathrm{CS}_{3,2}$, each of order 4. In this configuration, $\mathrm{CS}_{3,0}$ runs in free-running mode, while the other two MRMMs act as scramblers.

In all three cascade systems, the first MRMM ($\mathrm{CS}_{k,0}$) runs in free-running mode for $k = 1, 2, 3$. Since primitive MRMMs exhibit strong randomness properties, all MRMMs used in this experimental study are chosen to be primitive. Furthermore, all MRMMs are defined over the finite field $\mathbb{F}_2^{16}$, i.e., the word size is fixed at $m = 16$.

Let R_a be the right shift operator defined as $R_a(X) = X >> a$ whereas L_b be the left shift operator defined as $L_b(X) = X << b$ and the left circular shift of X by c-bit is denoted as $LCS_c(X)$. In our experimental set up, the matrix polynomial of $\mathrm{CS}_{1,0}$ is $x^{12} + L_9x^{11} + 48920x^7 + 34339x^5 + LCS_2$. The matrix polynomials used for $\mathrm{CS}_{2,0}$ and $\mathrm{CS}_{2,1}$ are respectively, $x^6 + (L_9 + R_4)x^1 + L_{14}x^3 + R_{11}x^2 + (L_1 + R_1)$ and $x^6 + L_3x^3 + R_{12}x^4 + L_8x + LCS_5$. Similarly, three matrix polynomials $x^4 + (L_3 + R_{10})x^3 + LCS_{13}$, $x^4 + (L_9 + R_6)x^3 + (L_{13} + R_3)$ and $x^4 + (L_3 + R_2)x + LCS_3$ are used for $\mathrm{CS}_{3,0}$, $\mathrm{CS}_{3,1}$ and $\mathrm{CS}_{3,1}$ respectively.

We avoid all zero-state initialization situations in $\mathrm{CS}_{\mathrm{k},0}$, otherwise it will be always in all zero states position. In $\mathbf{CS}_2$, it is observed that the results of the statistical test suite are similar for several random initializations of the states of $\mathrm{CS}_{2,1}$, including all zero initialization states. This occurs as the states of $\mathrm{CS}_{2,0}$ are injected into the states of $\mathrm{CS}_{2,1}$ through its feedback calculation. Similarly in $\mathbf{CS}_3$, the states of $\mathrm{CS}_{3,0}$ are injected into the states of $\mathrm{CS}_{3,1}$ and $\mathrm{CS}_{3,2}$. Thus, the states of MRMM except the free running MRMM in $\mathbf{CS}_2$ and $\mathbf{CS}_3$ can be initialized with any random values, but for our experimental set up we always initialize with zero values. Again, all states of $\mathrm{CS}_{k,0}$ are also initialized with zero except the first two states. These two states are initialized with a random 16-bit value $\mathbf{v}$ and its bitwise complement, i.e., $\mathbf{v} \oplus 0\mathrm{xFFFF}$. Thus, $\mathrm{CS}_{k0}[0] = \mathbf{v}$ and $\mathrm{CS}_{k0}[1] = \mathbf{v} \oplus 0\mathrm{xFFFF}$, for $k = 1, 2, 3$. Remaining states of $\mathbf{CS}_k$ are initialized to zero. Thus, the first two states are balanced i.e., number of 0s and 1s are equal. The other states become non-zero after a few iterations. Since the total number of 16-bit numbers is 2^{16}, the

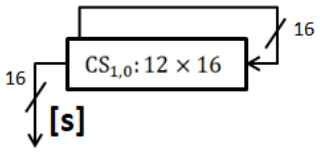


Figure 3: $\mathbf{CS}_1$.

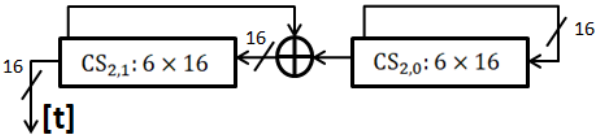


Figure 4: $\mathbf{CS}_2$.

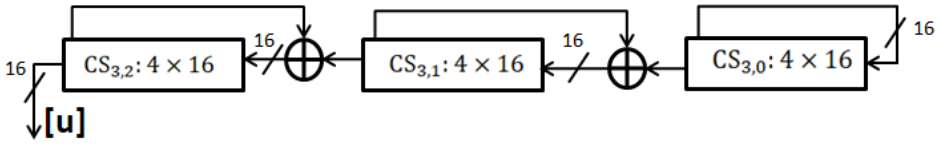


Figure 5: $\mathbf{CS}_3$.

experiment is conducted for all possible initializations for each system. We begin by studying the avalanche property of $\mathbf{CS}_1$.

4.1.1 Avalanche analysis on state registers of $\mathbf{CS}_1$

To compute the number of iterations required to achieve the avalanche property, the following steps are performed:

1. Set $count = 0$.
2. Initialize the states of $\mathbf{CS}_1$: Let $\mathbf{ini\text{-}s}_i$, for $0 \leq i \leq 11$, denote the initial values of the states of $\mathbf{CS}_1$. Set $\mathbf{ini\text{-}s}_0 = \mathbf{v}$, $\mathbf{ini\text{-}s}_1 = \mathbf{v}^c$, and remaining ten $\mathbf{ini\text{-}s}_1 = 0$. Now initialize $\mathrm{CS}_{10}[i] = \mathbf{ini\text{-}s}_i$.
3. Run $\mathbf{CS}_1$ for one iteration. Here one iteration means it calculates the feedback value fb_1 of CS_{10}, then shift the states i.e., $\mathrm{CS}_{10}[k] = \mathrm{CS}_{10}[k{+}1]$ for $0 \leq k < 11$ and $\mathrm{CS}_{10}[11] = fb_1$.
4. Calculate $weight = \sum_{k=0}^{11} Wt(\mathrm{CS}_{10}[k] \oplus \mathbf{ini\text{-}s}_k)$. Here $Wt(\boldsymbol{x})$ is the hamming weight of the word $\boldsymbol{x}$.
5. If $\mid weight - 96 \mid > 2$, increase $count$ by 1 and go to Step 3. Otherwise, return $count$. Note that 96 is the 50% of the total memory bit size of $\mathbf{CS}_1$, i.e., $mn = 192$, whereas 2 is the 1% approximation value of mn.

The experiment is conducted for every possible 16-bit value $\mathbf{v}$, resulting in a total of $2^{16} = 65536$ iterations. For each value of $\mathbf{v}$, the number of iterations k required to achieve the avalanche property is recorded. Figure 6 presents a plot of the number of 16-bit values $\mathbf{v}$ versus the corresponding iteration count k. The X-axis represents the number of iterations needed to achieve the avalanche property, while the Y-axis indicates the number of 16-bit values. The results reveal that, for the majority of inputs, the avalanche property is achieved after 60 iterations.

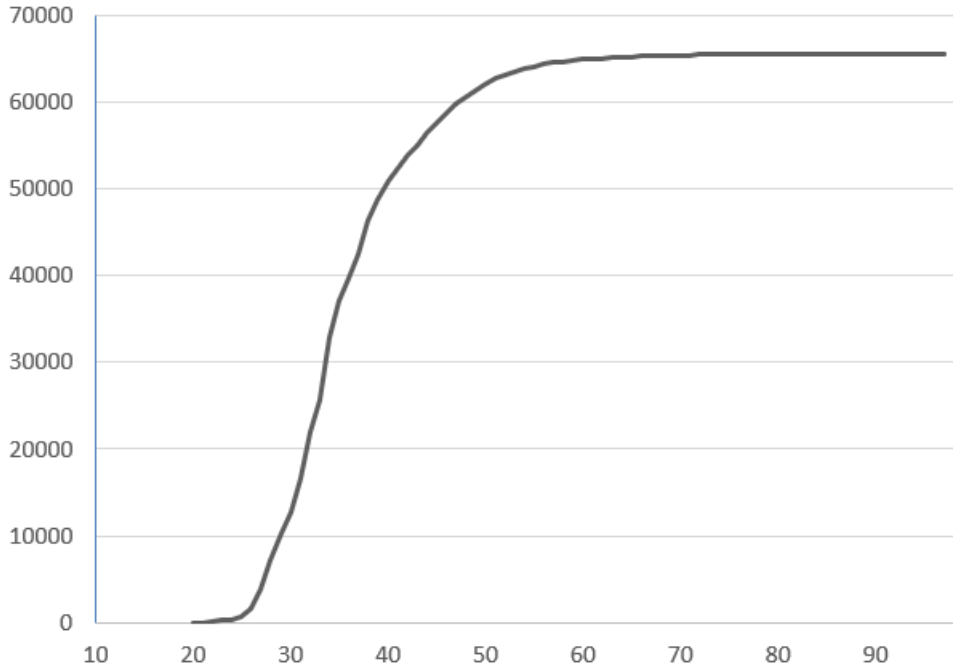


Figure 6: Avalanche analysis of $\mathbf{CS}_1$.

However, a small subset of values requires up to 97 iterations to achieve the same property.

4.1.2 Avalanche analysis on state registers of $\mathbf{CS}_2$ and $\mathbf{CS}_3$

In $\mathbf{CS}_1$, there was one MRMM whereas in $\mathbf{CS}_2$, there are two MRMMs. The procedure of calculating the number of iteration to achieve 50% avalanche is similar as in $\mathbf{CS}_1$. The procedure is as follows:

1. Set $count = 0$.
2. Initialize the states of $\mathbf{CS}_2$: Let $\mathbf{ini\text{-}s}_i$, for $0 \leq i \leq 11$, be the initial value of the states of $\mathbf{CS}_1$, where $\mathbf{ini\text{-}s}_0 = \mathbf{v}$, $\mathbf{ini\text{-}s}_1 = \mathbf{v}^c$, and remaining ten $\mathbf{ini\text{-}s}_1 = 0$. Now initialize $\mathrm{CS}_{20}[i] = \mathbf{ini\text{-}s}_i$ and $\mathrm{CS}_{21}[i] = \mathbf{ini\text{-}s}_{6+i}$.
3. Run $\mathbf{CS}_2$ for one iteration. One iteration in $\mathbf{CS}_2$ means it calculates the feedback value fb_2 of CS_{21}, then shift the states of CS_{21} and $\mathrm{CS}_{21}[5] = fb_2 \oplus \mathrm{CS}_{20}[0]$. It also computes the feedback value fb_1 of CS_{20}, then shift the states of CS_{20} and $\mathrm{CS}_{20}[5] = fb_1$.
4. Calculate $weight = \sum_{k=0}^{5}[Wt(\mathrm{CS}_{10}[k] \oplus \mathbf{ini\text{-}s}_k) + Wt(\mathrm{CS}_{10}[k] \oplus \mathbf{ini\text{-}s}_k)]$.
5. If $\mid weight - 96 \mid > 2$, increase $count$ by 1 and go to Step 3. Otherwise, return $count$.

In $\mathbf{CS}_3$, three MRMMs are employed. The procedure for determining the number of iterations required to achieve the 50% avalanche effect is similar to that used for $\mathbf{CS}_2$, with the primary difference being in the feedback computations. As in the case of $\mathbf{CS}_1$, the experiment is conducted for all possible 16-bit input values $\mathbf{v}$ for both $\mathbf{CS}_2$ and $\mathbf{CS}_3$.

A comparative summary of the results for these systems is presented in Table 3. The table shows that after 20 iterations, the percentage of 16-bit numbers achieving the avalanche effect is nearly zero for $\mathbf{CS}_1$, whereas it reaches 90.53% and 98.38% for $\mathbf{CS}_2$ and $\mathbf{CS}_3$, respectively. As illustrated in Figure 6 and Table 3, the avalanche property is attained more rapidly in $\mathbf{CS}_3$ compared to $\mathbf{CS}_1$ and $\mathbf{CS}_2$.

No. of Iterations to achieve avalanche criterion	**% of 16-bit** v for CS_1	**% of 16-bit** v for CS_2	**% of 16-bit** v for CS_3
10	0	15.5	66.01
15	0	63.31	92.46
20	0.01	90.53	98.38
25	1.11	97.58	99.62
30	19.38	99.37	99.92
35	56.7	99.85	99.98
40	77.64	99.96	100

Table 3: Avalanche analysis for $\mathbf{CS}_1$, $\mathbf{CS}_2$ and $\mathbf{CS}_3$.

5 Conclusion

In this paper, we investigated two configurations of LFSR-based systems and examined their periodic behavior in detail. We showed that cascaded LFSRs are a specific instance of the first configuration and derived an exact expression for their period under defined parameters. In the second configuration, we introduced carry bits as nonlinear components in the feedback calculation of the second LFSR and analyzed their periodicity. This structure was further generalized, revealing a deeper connection between LFSRs and Lagged Fibonacci Generators (LFGs). A key insight from our study is that LFGs can be effectively expressed in terms of simpler LFSR components. This decomposition offers tangible advantages in resource-constrained environments such as embedded systems, IoT devices, and sensor networks, where

low word sizes (typically 8 or 16 bits) make traditional LFG implementations impractical. By using multiple LFSRs with a shared feedback polynomial and integrating a small-size adder, we can replicate the behavior of an LFG using only basic shift and XOR operations. For instance, with a k-bit adder, an additive LFG (ALFG) with upto 2^k tap points can be efficiently realized, for any word size m. This LFSR-based approach paves the way for scalable, high-speed, and low-complexity pseudorandom number generators—particularly valuable for embedded cryptographic systems that demand both compactness and performance. Future work can explore the cryptographic strength, resistance to attacks, and hardware optimizations of these designs, potentially contributing to the development of next-generation lightweight stream ciphers. In the LFSR-based cascade system of the second configuration type, we have considered only the carry bit as the nonlinear function. Extending these results to a more general setting involving NFSRs would require substantially different analytic techniques. This forms an interesting direction for future research.

In this paper, we present compelling experimental results demonstrating the excellent avalanche properties of three MRMM-based cascade systems. Our findings reveal that, with careful selection of MRMMs, the resulting periods across all configurations remain comparable. Remarkably, the cascade systems not only maintain strong periodic behavior but also exhibit significantly enhanced linear complexity and achieve a faster avalanche effect compared to standalone MRMMs. These results clearly underscore the effectiveness of FSR cascading in significantly boosting the randomness characteristics of the system, making it a powerful technique in the design of secure and efficient cryptographic primitives.

6 Acknowledgment

The second author sincerely thanks the Director of CAIR for his continuous encouragement and support.

References

[1] S. Goonatilake, *Toward a Global Science: Mining Civilizational Knowledge*. Race, gender, and science, Indiana University Press, 1998.

[2] D. E. Knuth, *The art of computer programming, volume 2 (3rd ed.): seminumerical algorithms*. USA: Addison-Wesley Longman Publishing Co., Inc., 1997.

[3] R. P. Brent, "On the periods of generalized fibonacci recurrences," *Mathematics of Computation*, vol. 63, no. 207, pp. 389–401, 1994.

[4] M. K. Chetry, S. K. Bishoi, and V. Matyas, "When lagged fibonacci generators jump," *Discrete Applied Mathematics*, vol. 267, pp. 64–72, 2019.

[5] M. Mascagni and A. Srinivasan, "Algorithm 806: Sprng: a scalable library for pseudorandom number generation," *ACM Trans. Math. Softw.*, vol. 26, p. 436–461, Sept. 2000.

[6] S. K. Bishoi, H. K. Haran, and S. U. Hasan, "A note on the multiple-recursive matrix method for generating pseudorandom vectors," *Discrete Applied Mathematics*, vol. 222, pp. 67–75, 2017.

[7] H. Niederreiter, "The multiple-recursive matrix method for pseudorandom number generation," *Finite Fields and Their Applications*, vol. 1, no. 1, pp. 3–30, 1995.

[8] S. W. Golomb, "Shift register sequences – a retrospective account," in *Sequences and Their Applications – SETA 2006* (G. Gong, T. Helleseth, H.-Y. Song, and K. Yang, eds.), (Berlin, Heidelberg), pp. 1–4, Springer Berlin Heidelberg, 2006.

[9] A. J. Menezes, P. C. Van Oorschot, and S. A. Vanstone, *Handbook of applied cryptography*. CRC press, 2018.

[10] S. Bishoi, K. Senapati, and B. Shankar, "Generalized word-oriented feedback shift registers," *Cryptology ePrint Archive*, 2023.

[11] R. Lidl and H. Niederreiter, *Finite Fields and Their Applications*, vol. 20 of *Encyclopedia Of Mathematics And Its Applications*. Cambridge University Press, 1996.

[12] D. J. Bernstein, "CAESAR competition for authenticated encryption: Security, applicability, and robustness." `http://competitions.cr.yp.to/caesar.html`, 2013.

[13] "eSTREAM: The ecrypt stream cipher project." `http://competitions.cr.yp.to/estream.html`, 2008.

[14] G. Marsaglia and L.-H. Tsay, "Matrices and the structure of random number sequences," *Linear Algebra and its Applications*, vol. 67, pp. 147–156, 1985.

[15] H. Niederreiter, "Factorization of polynomials and some linear-algebra problems over finite fields," *Linear Algebra and its Applications*, vol. 192, pp. 301–328, 1993.

[16] H. Niederreiter, "Improved bounds in the multiple-recursive matrix method for pseudorandom number and vector generation," *Finite Fields and Their Applications*, vol. 2, no. 3, pp. 225–240, 1996.

[17] G. Zeng, W. Han, and K. He, "High efficiency feedback shift register: $\sigma-$LFSR." Cryptology ePrint Archive, Paper 2007/114, 2007.

[18] S. Ghorpade, S. Hasan, and M. Kumari, "Primitive polynomials, singer cycles and word-oriented linear feedback shift registers," *Designs Codes and Cryptography*, vol. 58, pp. 123–134, 02 2011.

[19] H. HU and G. GONG, "Periods on two kinds of nonlinear feedback shift registers with time varying feedback functions," *International Journal of Foundations of Computer Science*, vol. 22, no. 06, pp. 1317–1329, 2011.

[20] D. Green and K. Dimond, "Nonlinear product-feedback shift registers," *Proceedings of the Institution of Electrical Engineers*, vol. 117, pp. 681–686, 1970.

[21] S. Bishoi and V. Matyas, *MRMM-Based Keystream Generators for Information Security*, pp. 209–233. Springer Tracts in Electrical and Electronics Engineering, Springer, Singapore, 04 2024.